



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Examen Parcial I: Solucionario Oficial de Referencia (MedTriage Express)

Este documento contiene la solución completa de referencia técnica para el **Examen Parcial I** (Módulos 1 a 4). Incluye el código fuente completo del backend en Spring Boot 3 / versión de Java disponible en su computadora (`medtriage-backend`), la prueba unitaria en Mockito, el cliente SPA en Angular 19 Standalone (`medtriage-frontend`) y la guía de evaluación.

Parte 1: Base de Datos en SQL Server (`schema.sql`)

```
CREATE DATABASE MedTriageDB_<carnetestudiante>;
GO

USE MedTriageDB_<carnetestudiante>;
GO

CREATE TABLE Doctor (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
```

```

    nombre VARCHAR(100) NOT NULL,
    especialidad VARCHAR(80) NOT NULL,
    disponible BIT NOT NULL DEFAULT 1
);

CREATE TABLE Paciente (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    identificacion VARCHAR(20) NOT NULL UNIQUE,
    nombre_completo VARCHAR(120) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    telefono VARCHAR(20) NOT NULL
);

CREATE TABLE CitaMedica (
    id BIGINT IDENTITY(1,1) PRIMARY KEY,
    codigo_cita VARCHAR(30) NOT NULL UNIQUE,
    paciente_id BIGINT NOT NULL,
    doctor_id BIGINT NOT NULL,
    nivel_prioridad VARCHAR(20) NOT NULL,
    motivo_consulta VARCHAR(255) NOT NULL,
    estado VARCHAR(20) NOT NULL,
    monto_consulta DECIMAL(10,2) NOT NULL,
    fecha_cita DATETIME2 NOT NULL,
    CONSTRAINT FK_Cita_Paciente
        FOREIGN KEY (paciente_id) REFERENCES Paciente(id),
    CONSTRAINT FK_Cita_Doctor
        FOREIGN KEY (doctor_id) REFERENCES Doctor(id)
);
GO

```

Parte 2: Backend Spring Boot (medtriage-backend)

1. Entidades JPA (com.medtriage.model)

Doctor.java:

```

package com.medtriage.model;

import jakarta.persistence.*;

@Entity

```

```

@Table(name = "Doctor")
public class Doctor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, length = 80)
    private String especialidad;

    @Column(nullable = false)
    private Boolean disponible;

    public Doctor() {}

    public Doctor(String nombre,
                  String especialidad,
                  Boolean disponible) {
        this.nombre = nombre;
        this.especialidad = especialidad;
        this.disponible = disponible;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getEspecialidad() { return especialidad; }
    public void setEspecialidad(String especialidad) {
        this.especialidad = especialidad;
    }

    public Boolean getDisponible() { return disponible; }
    public void setDisponible(Boolean disponible) {
        this.disponible = disponible;
    }
}

```

Paciente.java:

```

package com.medtriage.model;

import jakarta.persistence.*;

@Entity
@Table(name = "Paciente")
public class Paciente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 20)
    private String identificacion;

    @Column(name = "nombre_completo", nullable = false)
    private String nombreCompleto;

    @Column(nullable = false, length = 100)
    private String correo;

    @Column(nullable = false, length = 20)
    private String telefono;

    public Paciente() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getIdentificacion() { return identificacion; }
    public void setIdentificacion(String identificacion) {
        this.identificacion = identificacion;
    }

    public String getNombreCompleto() { return nombreCompleto; }
    public void setNombreCompleto(String nombreCompleto) {
        this.nombreCompleto = nombreCompleto;
    }

    public String getCorreo() { return correo; }
    public void setCorreo(String correo) {
        this.correo = correo;
    }

    public String getTelefono() { return telefono; }
    public void setTelefono(String telefono) {

```

```
        this.telefono = telefono;
    }
}
```

CitaMedica.java:

```
package com.medtrriage.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "CitaMedica")
public class CitaMedica {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "codigo_cita", nullable = false, unique = true)
    private String codigoCita;

    @ManyToOne(optional = false)
    @JoinColumn(name = "paciente_id")
    private Paciente paciente;

    @ManyToOne(optional = false)
    @JoinColumn(name = "doctor_id")
    private Doctor doctor;

    @Column(name = "nivel_prioridad", nullable = false)
    private String nivelPrioridad;

    @Column(name = "motivo_consulta", nullable = false)
    private String motivoConsulta;

    @Column(nullable = false)
    private String estado;

    @Column(name = "monto_consulta", nullable = false)
    private Double montoConsulta;

    @Column(name = "fecha_cita", nullable = false)
    private LocalDateTime fechaCita;
}
```

```

    public CitaMedica() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getCodigoCita() { return codigoCita; }
    public void setCodigoCita(String codigoCita) {
        this.codigoCita = codigoCita;
    }

    public Paciente getPaciente() { return paciente; }
    public void setPaciente(Paciente paciente) {
        this.paciente = paciente;
    }

    public Doctor getDoctor() { return doctor; }
    public void setDoctor(Doctor doctor) {
        this.doctor = doctor;
    }

    public String getNivelPrioridad() { return nivelPrioridad; }
    public void setNivelPrioridad(String nivelPrioridad) {
        this.nivelPrioridad = nivelPrioridad;
    }

    public String getMotivoConsulta() { return motivoConsulta; }
    public void setMotivoConsulta(String motivoConsulta) {
        this.motivoConsulta = motivoConsulta;
    }

    public String getEstado() { return estado; }
    public void setEstado(String estado) {
        this.estado = estado;
    }

    public Double getMontoConsulta() { return montoConsulta; }
    public void setMontoConsulta(Double montoConsulta) {
        this.montoConsulta = montoConsulta;
    }

    public LocalDateTime getFechaCita() { return fechaCita; }
    public void setFechaCita(LocalDateTime fechaCita) {
        this.fechaCita = fechaCita;
    }
}

```

2. DTOs (com.medtriage.dto)

CitaMedicaDTO.java:

```
package com.medtriage.dto;

import java.time.LocalDateTime;

public class CitaMedicaDTO {
    private Long id;
    private String codigoCita;
    private String nombrePaciente;
    private String nombreDoctor;
    private String nivelPrioridad;
    private String motivoConsulta;
    private String estado;
    private Double montoConsulta;
    private LocalDateTime fechaCita;

    public CitaMedicaDTO() {}

    public CitaMedicaDTO(Long id, String codigoCita,
                          String nombrePaciente, String nombreDoctor,
                          String nivelPrioridad, String motivoConsulta,
                          String estado, Double montoConsulta,
                          LocalDateTime fechaCita) {
        this.id = id;
        this.codigoCita = codigoCita;
        this.nombrePaciente = nombrePaciente;
        this.nombreDoctor = nombreDoctor;
        this.nivelPrioridad = nivelPrioridad;
        this.motivoConsulta = motivoConsulta;
        this.estado = estado;
        this.montoConsulta = montoConsulta;
        this.fechaCita = fechaCita;
    }

    public Long getId() { return id; }
    public String getCodigoCita() { return codigoCita; }
    public String getNombrePaciente() { return nombrePaciente; }
    public String getNombreDoctor() { return nombreDoctor; }
    public String getNivelPrioridad() { return nivelPrioridad; }
    public String getMotivoConsulta() { return motivoConsulta; }
    public String getEstado() { return estado; }
```

```
    public Double getMontoConsulta() { return montoConsulta; }  
    public LocalDateTime getFechaCita() { return fechaCita; }  
}
```

CrearCitaDTO.java:

```
package com.medtriage.dto;  
  
public class CrearCitaDTO {  
    private Long pacienteId;  
    private Long doctorId;  
    private String nivelPrioridad;  
    private String motivoConsulta;  
    private Double montoConsulta;  
  
    public CrearCitaDTO() {}  
  
    public Long getPacienteId() { return pacienteId; }  
    public void setPacienteId(Long pacienteId) {  
        this.pacienteId = pacienteId;  
    }  
  
    public Long getDoctorId() { return doctorId; }  
    public void setDoctorId(Long doctorId) {  
        this.doctorId = doctorId;  
    }  
  
    public String getNivelPrioridad() { return nivelPrioridad; }  
    public void setNivelPrioridad(String nivelPrioridad) {  
        this.nivelPrioridad = nivelPrioridad;  
    }  
  
    public String getMotivoConsulta() { return motivoConsulta; }  
    public void setMotivoConsulta(String motivoConsulta) {  
        this.motivoConsulta = motivoConsulta;  
    }  
  
    public Double getMontoConsulta() { return montoConsulta; }  
    public void setMontoConsulta(Double montoConsulta) {  
        this.montoConsulta = montoConsulta;  
    }  
}
```

3. Repositorios (com.medtriage.repository)

```
package com.medtriage.repository;

import com.medtriage.model.CitaMedica;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface CitaMedicaRepository
    extends JpaRepository<CitaMedica, Long> {
    List<CitaMedica> findByNivelPrioridadOrderByFechaCitaDesc(
        String nivelPrioridad);
}
```

4. Capa de Servicio (com.medtriage.service)

```
package com.medtriage.service;

import com.medtriage.dto.CitaMedicaDTO;
import com.medtriage.dto.CrearCitaDTO;
import com.medtriage.model.*;
import com.medtriage.repository.*;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

@Service
public class CitaMedicaServiceImpl implements CitaMedicaService {

    private final CitaMedicaRepository citaRepository;
    private final PacienteRepository pacienteRepository;
    private final DoctorRepository doctorRepository;

    public CitaMedicaServiceImpl(CitaMedicaRepository citaRepository,
                                PacienteRepository pacienteRepository,
```

```

        DoctorRepository doctorRepository) {
    this.citaRepository = citaRepository;
    this.pacienteRepository = pacienteRepository;
    this.doctorRepository = doctorRepository;
}

@Override
@Transactional(readOnly = true)
public List<CitaMedicaDTO> obtenerTodas() {
    return citaRepository.findAll()
        .stream()
        .map(this::convertirADTO)
        .collect(Collectors.toList());
}

@Override
@Transactional(readOnly = true)
public List<CitaMedicaDTO> obtenerPorPrioridad(String prioridad) {
    return citaRepository
        .findByNivelPrioridadOrderByFechaCitaDesc(prioridad)
        .stream()
        .map(this::convertirADTO)
        .collect(Collectors.toList());
}

@Override
@Transactional
public CitaMedicaDTO crearCita(CrearCitaDTO dto) {
    Paciente paciente = pacienteRepository.findById(dto.getPacienteId())
        .orElseThrow(() -> new RuntimeException(
            "Paciente no encontrado con ID: " + dto.getPacienteId()));

    Doctor doctor = doctorRepository.findById(dto.getDoctorId())
        .orElseThrow(() -> new RuntimeException(
            "Doctor no encontrado con ID: " + dto.getDoctorId()));

    if (!doctor.getDisponible()) {
        throw new IllegalStateException(
            "El doctor " + doctor.getNombre() + " no está disponible.");
    }

    CitaMedica cita = new CitaMedica();
    cita.setCodigoCita("CIT-2026-" + UUID.randomUUID()
        .toString().substring(0, 4).toUpperCase());
    cita.setPaciente(paciente);
    cita.setDoctor(doctor);
}

```

```

        cita.setNivelPrioridad(dto.getNivelPrioridad());
        cita.setMotivoConsulta(dto.getMotivoConsulta());
        cita.setEstado("PENDIENTE");
        cita.setMontoConsulta(dto.getMontoConsulta());
        cita.setFechaCita(LocalDate.now());

        CitaMedica guardada = citaRepository.save(cita);
        return convertirADTO(guardada);
    }

    @Override
    @Transactional
    public CitaMedicaDTO actualizarEstado(Long id, String nuevoEstado) {
        CitaMedica cita = citaRepository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Cita no encontrada con ID: " + id));

        cita.setEstado(nuevoEstado);
        return convertirADTO(citaRepository.save(cita));
    }

    private CitaMedicaDTO convertirADTO(CitaMedica c) {
        return new CitaMedicaDTO(
            c.getId(),
            c.getCodigoCita(),
            c.getPaciente().getNombreCompleto(),
            c.getDoctor().getNombre(),
            c.getNivelPrioridad(),
            c.getMotivoConsulta(),
            c.getEstado(),
            c.getMontoConsulta(),
            c.getFechaCita()
        );
    }
}

```

5. Manejador de Excepciones RFC 7807 (GlobalExceptionHandler.java)

```

package com.medtrriage.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;

```

```

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalStateException.class)
    public ResponseEntity<ProblemDetail> handleIllegalState(
        IllegalStateException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, ex.getMessage());
        problem.setTitle("Regla de Negocio Violada");
        problem.setType(URI.create("https://medtriage.ucr.ac.cr/errors/business-rule"));
        return ResponseEntity.badRequest().body(problem);
    }
}

```

6. Controlador RESTful (CitaMedicaController.java)

```

package com.medtriage.controller;

import com.medtriage.dto.CitaMedicaDTO;
import com.medtriage.dto.CrearCitaDTO;
import com.medtriage.service.CitaMedicaService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/v1/citas")
@CrossOrigin(origins = "http://localhost:4200")
public class CitaMedicaController {

    private final CitaMedicaService citaService;

    public CitaMedicaController(CitaMedicaService citaService) {
        this.citaService = citaService;
    }
}

```

```

    @GetMapping
    public ResponseEntity<List<CitaMedicaDTO>> obtenerTodas() {
        return ResponseEntity.ok(citaService.obtenerTodas());
    }

    @GetMapping("/prioridad/{prioridad}")
    public ResponseEntity<List<CitaMedicaDTO>> obtenerPorPrioridad(
        @PathVariable String prioridad) {
        return ResponseEntity.ok(
            citaService.obtenerPorPrioridad(prioridad));
    }

    @PostMapping
    public ResponseEntity<CitaMedicaDTO> crearCita(
        @RequestBody CrearCitaDTO dto) {
        return ResponseEntity.ok(citaService.crearCita(dto));
    }

    @PatchMapping("/{id}/estado")
    public ResponseEntity<CitaMedicaDTO> actualizarEstado(
        @PathVariable Long id,
        @RequestBody Map<String, String> payload) {
        String estado = payload.get("estado");
        return ResponseEntity.ok(
            citaService.actualizarEstado(id, estado));
    }
}

```

7. Prueba Unitaria con JUnit 5 y Mockito (CitaMedicaServiceTest.java)

```

package com.medtriage.service;

import com.medtriage.dto.CrearCitaDTO;
import com.medtriage.model.*;
import com.medtriage.repository.*;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

```

```

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class CitaMedicaServiceTest {

    @Mock
    private CitaMedicaRepository citaRepository;
    @Mock
    private PacienteRepository pacienteRepository;
    @Mock
    private DoctorRepository doctorRepository;

    @InjectMocks
    private CitaMedicaServiceImpl citaService;

    @Test
    void crearCitaExitosamente() {
        CrearCitaDTO dto = new CrearCitaDTO();
        dto.setPacienteId(1L);
        dto.setDoctorId(1L);
        dto.setNivelPrioridad("ALTA");
        dto.setMotivoConsulta("Urgencia respiratoria");
        dto.setMontoConsulta(25000.0);

        Paciente paciente = new Paciente();
        paciente.setId(1L);
        paciente.setNombreCompleto("Elena Madrigal");

        Doctor doctor = new Doctor("Dr. Carlos", "General", true);
        doctor.setId(1L);

        when(pacienteRepository.findById(1L))
            .thenReturn(Optional.of(paciente));
        when(doctorRepository.findById(1L))
            .thenReturn(Optional.of(doctor));
        when(citaRepository.save(any(CitaMedica.class)))
            .thenReturn(invocation -> invocation.getArgument(0));

        var resultado = citaService.crearCita(dto);

        assertNotNull(resultado);
        assertEquals("ALTA", resultado.getNivelPrioridad());
    }
}

```

```
    assertEquals("PENDIENTE", resultado.getEstado());  
    verify(citaRepository, times(1)).save(any());  
  }  
}
```

Parte 3: Frontend Angular 19 (medtriage-frontend)

1. Modelo TypeScript (src/app/models/cita.model.ts)

```
export interface CitaMedica {  
  id: number;  
  codigoCita: string;  
  nombrePaciente: string;  
  nombreDoctor: string;  
  nivelPrioridad: 'ALTA' | 'MEDIA' | 'BAJA';  
  motivoConsulta: string;  
  estado: 'PENDIENTE' | 'EN_ATENCION' | 'ATENDIDA' | 'CANCELADA';  
  montoConsulta: number;  
  fechaCita: string;  
}  
  
export interface CrearCitaPayload {  
  pacienteId: number;  
  doctorId: number;  
  nivelPrioridad: string;  
  motivoConsulta: string;  
  montoConsulta: number;  
}
```

2. Servicio HTTP (src/app/services/cita.service.ts)

```
import { Injectable, inject } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { Observable } from 'rxjs';  
import { environment } from '../../../environments/environment';  
import { CitaMedica, CrearCitaPayload } from '../../../models/cita.model';
```

```

@Inject({
  providedIn: 'root'
})
export class CitaService {
  private http = inject(HttpClient);
  private apiUrl = `${environment.API_URL}citas`;

  getCitas(): Observable<CitaMedica[]> {
    return this.http.get<CitaMedica[]>(this.apiUrl);
  }

  getCitasPorPrioridad(prioridad: string): Observable<CitaMedica[]> {
    return this.http.get<CitaMedica[]>(
      `${this.apiUrl}/prioridad/${prioridad}`
    );
  }

  crearCita(payload: CrearCitaPayload): Observable<CitaMedica> {
    return this.http.post<CitaMedica>(this.apiUrl, payload);
  }

  actualizarEstado(id: number, estado: string): Observable<CitaMedica> {
    return this.http.patch<CitaMedica>(
      `${this.apiUrl}/${id}/estado`,
      { estado }
    );
  }
}

```

3. Componente Dashboard con Signals (CitaListComponent)

cita-list.component.ts:

```

import { Component, OnInit, signal, computed, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { CitaMedica } from '../../models/cita.model';
import { CitaService } from '../../services/cita.service';

@Component({
  selector: 'app-cita-list',
  standalone: true,
  imports: [CommonModule],

```

```

    templateUrl: './cita-list.component.html',
    styleUrls: ['./cita-list.component.css']
  })
  export class CitaListComponent implements OnInit {
    private citaService = inject(CitaService);

    // Angular Signals para estado local reactivo
    citasSignal = signal<CitaMedica[]>([]);
    prioridadFiltro = signal<string>('TODAS');

    // Computed Signal para filtrar automáticamente
    citasFiltradas = computed(() => {
      const filtro = this.prioridadFiltro();
      const lista = this.citasSignal();
      if (filtro === 'TODAS') return lista;
      return lista.filter(c => c.nivelPrioridad === filtro);
    });

    ngOnInit(): void {
      this.cargarCitas();
    }

    cargarCitas(): void {
      this.citaService.getCitas().subscribe(data => {
        this.citasSignal.set(data);
      });
    }

    filtrarPorPrioridad(prioridad: string): void {
      this.prioridadFiltro.set(prioridad);
    }

    avanzarEstado(cita: CitaMedica): void {
      let siguienteEstado = 'EN_ATENCION';
      if (cita.estado === 'EN_ATENCION') siguienteEstado = 'ATENDIDA';

      this.citaService.actualizarEstado(cita.id, siguienteEstado).subscribe(
        actualizada => {
          this.citasSignal.update(lista =>
            lista.map(c => c.id === actualizada.id ? actualizada : c)
          );
        }
      );
    }
  }

```

cita-list.component.html:

```
<div class="container">
  <h2>■ Panel de Triage y Citas Médicas</h2>
  <div class="filters">
    <button (click)="filtrarPorPrioridad('TODAS')"
      [class.active]="prioridadFiltro() === 'TODAS'">Todas</button>
    <button (click)="filtrarPorPrioridad('ALTA')"
      [class.active]="prioridadFiltro() === 'ALTA'">Prioridad Alta</button>
    <button (click)="filtrarPorPrioridad('MEDIA')"
      [class.active]="prioridadFiltro() === 'MEDIA'">Prioridad Media</button>
    <button (click)="filtrarPorPrioridad('BAJA')"
      [class.active]="prioridadFiltro() === 'BAJA'">Prioridad Baja</button>
  </div>
  <table class="table">
    <thead>
      <tr>
        <th>Código</th>
        <th>Paciente</th>
        <th>Doctor</th>
        <th>Prioridad</th>
        <th>Motivo</th>
        <th>Estado</th>
        <th>Acción</th>
      </tr>
    </thead>
    <tbody>
      @for (cita of citasFiltradas(); track cita.id) {
        <tr>
          <td><strong>{{ cita.codigoCita }}</strong></td>
          <td>{{ cita.nombrePaciente }}</td>
          <td>{{ cita.nombreDoctor }}</td>
          <td>
            <span class="badge" [class]="cita.nivelPrioridad.toLowerCase()">
              {{ cita.nivelPrioridad }}
            </span>
          </td>
          <td>{{ cita.motivoConsulta }}</td>
          <td>{{ cita.estado }}</td>
          <td>
            @if (cita.estado !== 'ATENDIDA' && cita.estado !== 'CANCELADA') {
              <button class="btn-action" (click)="avanzarEstado(cita)">
                Avanzar Estado
              </button>
            }
          </td>
        </tr>
      }
    </tbody>
  </table>
</div>
```

```
    }  
  </td>  
</tr>  
} @empty {  
  <tr>  
    <td colspan="7">No hay citas registradas en el sistema.</td>  
  </tr>  
}  
</tbody>  
</table>  
</div>
```

Verificación y Calificación de la Prueba

El evaluador debe ejecutar el backend en el puerto 8080 e inspeccionar el frontend en <http://localhost:4200/citas>. Verificar la correcta ejecución de la prueba unitaria Mockito (`mvn test`) y los commits individuales en GitHub.